ShunyaLabs

TTS API Integration Guide

Client integration guide for ShunyaLabs' latest Text-to-Speech endpoints, including standard TTS, education-focused LaTeX TTS, and OpenAI-compatible voice-agent / Pipecat integration.

New endpoints: These endpoints are the recommended integration path for new client implementations and are intended to provide an improved experience over the previous TTS endpoints.

Capability	Endpoint / Interface
Authentication	POST https://app.shunyalabs.ai/api/auth/token
Standard TTS	POST https://ttsv2.shunyalabs.ai/v1/omni-voice/synthesize
Education TTS	POST https://ttsv2.shunyalabs.ai/v1/omni-voice/synthesize
Voice Agent / Pipecat	POST https://ttsv2.shunyalabs.ai/v1/audio/speech
API Documentation	https://ttsv2.shunyalabs.ai/docs

Quick Start

1. Visit https://shunyalabs.ai and sign up or log in.
2. Complete the onboarding flow. Testing credits will be added to your account.
3. Generate an API key from your ShunyaLabs account.
4. Exchange the API key for an authentication token using the Auth API.
5. Use the returned token as a Bearer token when calling the TTS endpoints.

Security: Keep API keys and authentication tokens on the server side. Do not expose them in browser code, mobile app bundles, public repositories, or logs.

1. Authentication

ShunyaLabs TTS APIs use a short-lived Bearer token. Generate your API key from your ShunyaLabs account, then exchange it for a token using the authentication endpoint.

Generate a token

```
curl -X POST 'https://app.shunyalabs.ai/api/auth/token' \
-H 'api-key: YOUR_API_KEY' \
-H 'Content-Type: application/json' \
-d '{"expires_in": 86400}'
```

expires_in specifies the requested token lifetime in seconds. In the example above, the token is requested for 86,400 seconds (24 hours). Use the token returned by this API as Authorization: Bearer YOUR_TOKEN in subsequent requests.

Recommended flow

- Store the API key securely in your backend or secret manager.
- Request an auth token from your backend when required.
- Reuse the token until it approaches expiry, then obtain a fresh token.
- Never send your ShunyaLabs API key directly to end-user clients.

2. Standard Text-to-Speech

Use the Omni Voice synthesis endpoint for standard text-to-speech generation.

```
curl -X POST 'https://ttsv2.shunyalabs.ai/v1/omni-voice/synthesize' \
-H 'Authorization: Bearer YOUR_TOKEN' \
-H 'Content-Type: application/json' \
-d '{
"text": "If Modi is prime minister who is Amit Shah?",
"voice": "Meera"
}' \
--output speech.wav
```

Key fields: text contains the content to synthesize and voice selects the requested ShunyaLabs voice.

3. Education TTS with LaTeX

For educational content containing mathematical expressions, enable LaTeX processing and provide the language. This allows math-oriented content to be handled through the same Omni Voice synthesis endpoint.

```
curl -X POST 'https://ttsv2.shunyalabs.ai/v1/omni-voice/synthesize' \
-H 'Authorization: Bearer YOUR_TOKEN' \
-H 'Content-Type: application/json' \
-d '{
"text": "If $x^2 = 49$, what is the value of x?",
"voice": "Meera",
"latex": true,
"language": "en"
}' \
--output math.wav
```

Additional fields: Set latex to true when the input contains LaTeX expressions. Use language to identify the content language.

4. Voice Agent / OpenAI-Compatible TTS

For real-time voice-agent frameworks and integrations expecting an OpenAI-compatible speech interface, use the /v1/audio/speech endpoint.

```
curl -X POST 'https://ttsv2.shunyalabs.ai/v1/audio/speech' \
-H 'Authorization: Bearer YOUR_TOKEN' \
-H 'Content-Type: application/json' \
-d '{
"model": "tts-1",
"input": "□□□□□□□, □□□□ □□ □□□□□□ □□ □□□□ □□□□□□ □□□?",
"voice": "Meera",
"response_format": "pcm"
}' \
--output speech.pcm
```

The example requests raw PCM output, which is useful for streaming and voice-agent pipelines. Select the response format required by your integration as supported by the API.

Pipecat example

```
from pipecat.services.openai.tts import OpenAITTSService

tts = OpenAITTSService(
api_key="YOUR_TOKEN",
base_url="https://ttsv2.shunyalabs.ai/v1",
voice="nova",
)
```

> Pipecat note: The api_key field name comes from Pipecat's OpenAI-compatible service interface; pass the ShunyaLabs auth token as its value. For Pipecat, use an OpenAI-compatible voice name such as nova in the client configuration.

5. Client Integration Checklist

- Create or log in to your ShunyaLabs account at https://shunyalabs.ai.
- Complete onboarding and confirm that testing credits are available.
- Generate an API key.
- Exchange the API key for a Bearer token using the Auth API.
- Integrate Standard TTS, Education TTS, or the OpenAI-compatible endpoint as required.
- Store credentials securely and refresh tokens before expiry.
- Use the latest API documentation for supported request fields and integration details.

6. API Documentation

Interactive API documentation is available at:
https://ttsv2.shunyalabs.ai/docs

7. Before Production Rollout

The integration examples above are sufficient for initial testing. Before moving a client workload to production, we recommend confirming the production requirements relevant to that implementation, including expected traffic, required voices and languages, output/streaming format, latency expectations, and credit or commercial plan.

> Items worth confirming for each client: supported voices and languages, available response formats, rate/concurrency limits, production credit or billing plan, and the preferred support/escalation channel. These details are intentionally not hard-coded in this guide because they may vary by product version or client plan.

ShunyaLabs

Website: https://shunyalabs.ai
TTS API Docs: https://ttsv2.shunyalabs.ai/docs